

Remapping Keys with XKB and KLFC

Rohit Goswami · 2020-12-05 · workflow · tools · keyboard

An introduction to hacking keyboard layouts with X keyboard extension (XKB) and `klfc`, focused on Colemak and vim bindings

Background

In spite of maximizing ergonomic bindings for most common software (e.g. Vimium, doom-emacs), every operation with the arrow keys still trouble me. Here I will lay out my experiments transitioning to a stable, uniquely defined setup with the X keyboard extension.

Keyboard Basics

Some terms to keep in mind for this post are^[^fn:1]:

Dead Keys

These don't actually output anything, but modify the next key pressed. Like applying an umlaut on the subsequent letter.

Lock Keys

State modifiers which are toggled, like Caps Lock

Compose Key

A key which interprets a series of subsequent key strokes. A dead key on steroids.

Also the different levels ([from here](#)) are concisely defined in the following table.

Table 1: Levels for a keyboard

Level	Modifier	Keys
1	None	Lowercase letters, numbers other symbols
2	Shift	Uppercase letters, symbols placed above numbers
3	AltGr	Accented characters, symbols, some dead keys
4	Shift+AltGr	More dead keys and symbols
5	Extend	User-defined
6	Shift+Extend	User-defined

Modification Strategies

Common approaches to quick remapping of keys involves `xmodmap`, which does not persist between reboots. Manually recreating or spinning off of XKB configuration files was also not very appealing.

KLFC

A more elegant approach is by using the excellent [klfc Haskell binary](#). To install this from source:

```
git clone https://github.com/39aldo39/klfc
cd klfc
# Kanged from the AUR https://aur.archlinux.org/packages/klfc/
cabal v1-update
cabal v1-install --only-dependencies --ghc-options=-dynamic --force-reinstalls
cabal v1-configure --prefix=/usr --ghc-options=-dynamic
cabal v1-build
```

The output binary is in `./dist/build/klfc/klfc`.

Note that the set of keys mapped by the `json` files are relative to the QWERTY layout, that is:

```
= Tilde | N1 | N2 | N3 | N4 | N5 | N6 | N7 | N8 | N9 | N0 | Minus | Plus | Backspace
| Tab | Q | W | E | R | T | Y | U | I | O | P | Bracket_L | Bracket_R | Backslash
| CapsLock | A | S | D | F | G | H | J | K | L | Semicolon | Apostrophe | Enter
| Shift_L | Iso | Z | X | C | V | B | N | M | Comma | Period | Slash | Shift_R
| Control_L | Win_L | Alt_L | Space | Alt_R | Win_R | Menu | Control_R

| Esc | F1 | F2 | F3 | F4 | F5 | F6 | F7 | F8 | F9 | F10 | F11 | F12 | PrintScreen | ScrollLock | Pause
| Insert | Delete | Home | End | PageUp | PageDown | Up | Left | Down | Right

| NumLock | KP_Div | KP_Mult | KP_Min
| KP_7 | KP_8 | KP_9 | KP_Plus
| KP_4 | KP_5 | KP_6 | KP_Comma
| KP_1 | KP_2 | KP_3 | KP_Enter
| KP_0 | KP_Dec | KP_Eq

| AudioPlay | AudioPause | PlayPause | Previous | Next | Stop | ToggleRepeat | ToggleRandom
| AudioRewind | AudioForward
| Mute | VolumeDown | VolumeUp
| Eject

| Browser_Back | Browser_Forward | Browser_Refresh | Browser_Stop | Browser_Search | Browser_Favorites

| Calculator | MediaPlayer | Browser | Mail | Search | Explorer | WWW | MyComputer | Help
| Launch0 | Launch1 | Launch2 | Launch3 | Launch4 | Launch5 | Launch6 | Launch7
| Launch8 | Launch9 | LaunchA | LaunchB | LaunchC | LaunchD | LaunchE | LaunchF

| Power | Sleep | Wake | BrightnessDown | BrightnessUp

| F13 | F14 | F15 | F16 | F17 | F18 | F19 | F20 | F21 | F22 | F23 | F24
| Hash | Ro | Yen | Muhenkan | Henkan | Katakana
```

This means that we have to ensure that the keys are mapped relative to QWERTY as well, **not** relative to the modified base layout.

Remapping

Some goals were:

- Programming (particularly in `python` and `lisp`) put a lot of stress on the right hand pinky for Colemak users^[^fn:2]
- VIM keys should be global but toggled with a lock

My primary use case is currently my ThinkPad X380, which comes with a basic [QWERTY contracted layout](#) as shown in Fig. 1.



Figure 1: Basic X380 QWERTY

Colemak - Layers 1 and 2

The first mapping is a [basic Colemak setup](#) as shown in Fig. [fig:colemak](#).



Figure 2: Basic X380 Colemak

It wouldn't make much sense to remap the first two layers. We can use the `json` from the examples of the `klfc` repository.

```
// Base Colemak layout
// https://colemak.com
{
  "fullName": "Colemak",
  "name": "colemak",
  "localeId": "00000409",
  "copyright": "Public Domain",
  "company": "2006-01-01 Shai Coleman",
  "version": "1.0",
  "shiftlevels": [ "None", "Shift" ],
  "singletonKeys": [
    [ "CapsLock", "Backspace" ]
  ],
  "keys": [
    { "pos": "~", "letters": [ "`", "~" ] },
    { "pos": "1", "letters": [ "1", "!" ] },
    { "pos": "2", "letters": [ "2", "@" ] },
    { "pos": "3", "letters": [ "3", "#" ] },
    { "pos": "4", "letters": [ "4", "$" ] },
    { "pos": "5", "letters": [ "5", "%" ] },
    { "pos": "6", "letters": [ "6", "^" ] },
    { "pos": "7", "letters": [ "7", "&" ] },
    { "pos": "8", "letters": [ "8", "*" ] },
    { "pos": "9", "letters": [ "9", "(" ] },
    { "pos": "0", "letters": [ "0", ")" ] },
    { "pos": "-", "letters": [ "-", "_" ] },
    { "pos": "+", "letters": [ "=", "+" ] },
    { "pos": "Q", "letters": [ "q", "Q" ] },
    { "pos": "W", "letters": [ "w", "W" ] },
    { "pos": "E", "letters": [ "f", "F" ] },
    { "pos": "R", "letters": [ "p", "P" ] },
    { "pos": "T", "letters": [ "g", "G" ] },
    { "pos": "Y", "letters": [ "j", "J" ] },
    { "pos": "U", "letters": [ "l", "L" ] },
    { "pos": "I", "letters": [ "u", "U" ] },
    { "pos": "O", "letters": [ "y", "Y" ] },
    { "pos": "P", "letters": [ ";", ":" ] },
    { "pos": "[", "letters": [ "[", "{" ] },
    { "pos": "]", "letters": [ "]", "}" ] },
    { "pos": "\\\"", "letters": [ "\\\"", "|" ] },
    { "pos": "A", "letters": [ "a", "A" ] },
    { "pos": "S", "letters": [ "r", "R" ] },
    { "pos": "D", "letters": [ "s", "S" ] },
    { "pos": "F", "letters": [ "t", "T" ] },
    { "pos": "G", "letters": [ "d", "D" ] },
    { "pos": "H", "letters": [ "h", "H" ] },
    { "pos": "J", "letters": [ "n", "N" ] },
    { "pos": "K", "letters": [ "e", "E" ] },
    { "pos": "L", "letters": [ "i", "I" ] },
    { "pos": ";", "letters": [ "o", "O" ] },
    { "pos": "'", "letters": [ "'", "\"" ] },
    { "pos": "Z", "letters": [ "z", "Z" ] },
    { "pos": "X", "letters": [ "x", "X" ] },
    { "pos": "C", "letters": [ "c", "C" ] },
    { "pos": "V", "letters": [ "v", "V" ] },
    { "pos": "B", "letters": [ "b", "B" ] },
    { "pos": "N", "letters": [ "k", "K" ] },
    { "pos": "M", "letters": [ "m", "M" ] },
    { "pos": ",", "letters": [ ",", "<" ] },
    { "pos": ".", "letters": [ ".", ">" ] },
    { "pos": "/", "letters": [ "/", "?" ] }
  ],
  "variants": [
    {
      "name": "mod-dh",
    }
  ]
}
```

```

"shiftlevels": [ "None", "Shift" ],
"keys": [
  { "pos": "V", "letters": [ "d", "D" ] },
  { "pos": "B", "letters": [ "v", "V" ] },
  { "pos": "G", "letters": [ "g", "G" ] },
  { "pos": "T", "letters": [ "b", "B" ] },
  { "pos": "H", "letters": [ "k", "K" ] },
  { "pos": "N", "letters": [ "m", "M" ] },
  { "pos": "M", "letters": [ "h", "H" ] }
]
}
]
}

```

VIM Extensions

The additions are primarily through the Extend Layer[^{^fn:3}] (Fig. 3), with a Shift addition (Fig. 4) and more keys with AltGr (Fig. 5). As mentioned before, we have to continue mapping relative to QWERTY, so these mappings can be used by QWERTY users as well. We will essentially use the `ISO_5` shift key.

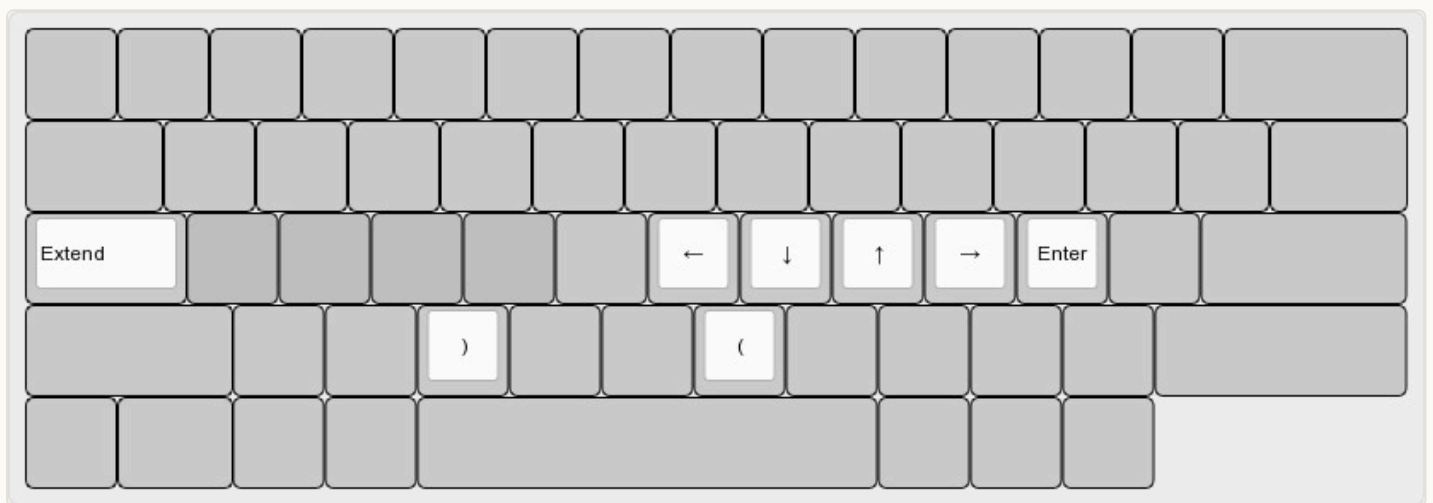


Figure 3: Extend layer mapping

- Maps basic `vim` movement

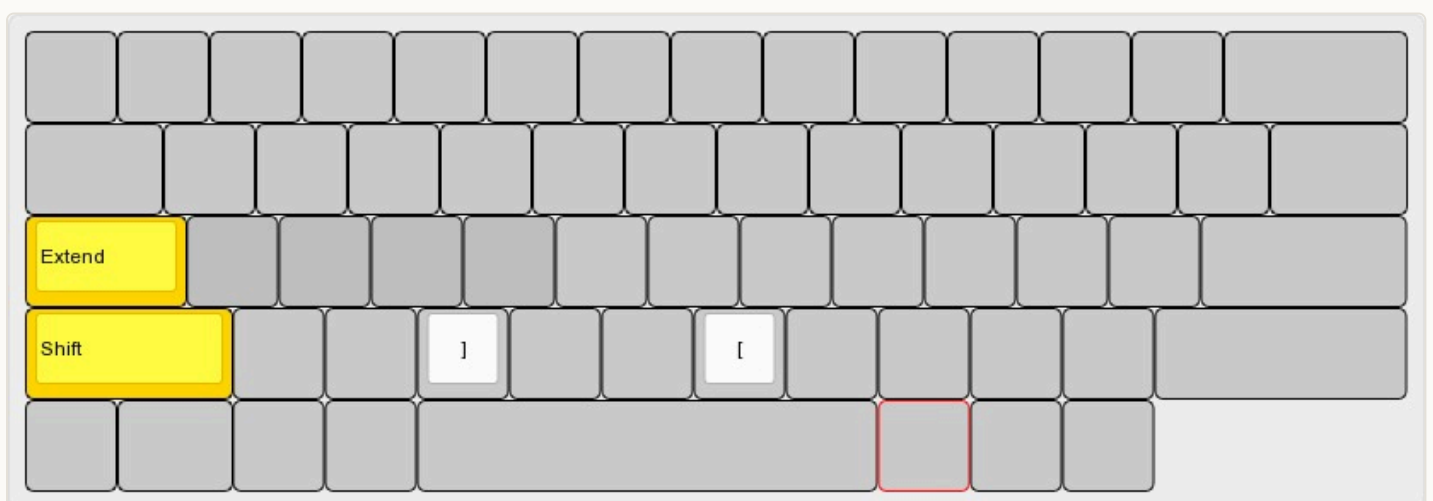


Figure 4: Extend+Shift layer mapping

- Relatively empty, just has a bracket

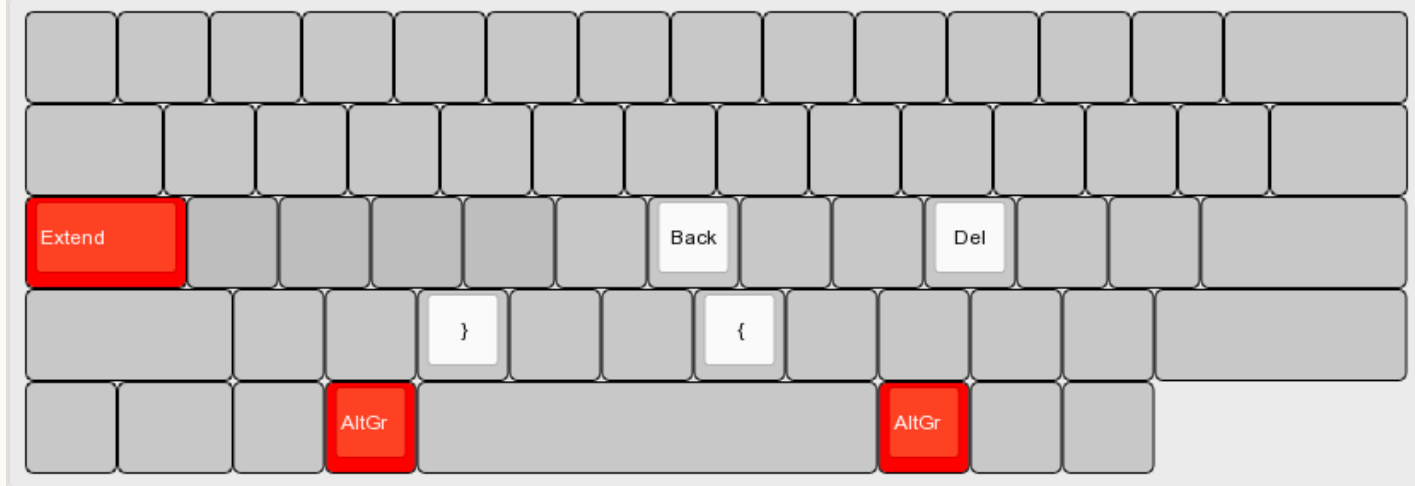


Figure 5: Extend+AltGr layer mapping

- Includes deletions

These are defined in a single `json` as shown.

```
{
  "filter": "no klc, keylayout",
  "singletonKeys": [
    [ "CapsLock", "Extend" ],
    [ "Alt_L", "AltGr" ]
  ],
  "shiftlevels": [ "Extend", "Shift+Extend", "AltGr+Extend" ],
  "keys": [
    { "pos": "H", "letters": [ "Left", "", "Backspace" ] },
    { "pos": "J", "letters": [ "Down" ] },
    { "pos": "K", "letters": [ "Up" ] },
    { "pos": "L", "letters": [ "Right", "", "Delete" ] },
    { "pos": ";", "letters": [ "Enter" ] },
    { "pos": "N", "letters": [ "(", "[", "{" ] },
    { "pos": "V", "letters": [ ")", "]", "}" ] }
  ]
}
```

- The idea is to have the `AltGr` keys placed symmetrically
- One major issue is that `Backspace` has gone from a single stroke of `CapsLock` to a three-key-combo
 - This is the least intuitive, and might need to be changed
- The third level (`Extend+AltGr`) is more accessible than the second in this layout

Usage

To generate the files needed to load the new layout:

```
klfc colemak.json extendVIM.json -o coleVIM
cd coleVIM/xkb
./run-session.sh # to try them out
./install-system.sh && ./scripts/install-xcompose.sh # to install them
```

Conclusions

The layout takes a bit of time to get used to, but it is a lot more transparent in the end compared to manually remapping to Colemak's NEIO instead of HJKL for movement. It is both persistent and easily extended, though it is likely that more needs to be done. Perhaps some metrics [^fn:4] might be collected as well.

